

11 — Query Execution Pipeline

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 55 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview](#)
 3. [Stage 1: Parser \(Raw Parse Tree\)](#)
 4. [Stage 2: Analyzer / Semantic Analysis \(Query Tree\)](#)
 5. [Stage 3: Rewriter \(Rules and Views\)](#)
 6. [Stage 4: Planner / Optimizer](#)
 - [Cost Estimation](#)
 - [Join Ordering](#)
 - [Index Selection](#)
 - [Key Planner Parameters](#)
 7. [Stage 5: Executor](#)
 8. [Plan Node Types](#)
 9. [EXPLAIN and EXPLAIN ANALYZE](#)
 10. [ASCII Pipeline Diagram](#)
 11. [Live Observation Queries](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises with Solutions](#)
 17. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe the five stages of the PostgreSQL query execution pipeline.
 - Explain what transformations happen at each stage and what data structures are produced.
 - Read and interpret `EXPLAIN` and `EXPLAIN ANALYZE` output accurately.
 - Identify the key planner GUC parameters and explain what enabling/disabling each changes.
 - Explain how the planner estimates costs and why wrong statistics lead to bad plans.
 - Describe the executor's pull-based (volcano/iterator) model.
-

Overview

Every SQL statement PostgreSQL executes goes through a 5-stage pipeline:

```
SQL Text
|
▼ Stage 1: Parser
Raw Parse Tree
|
▼ Stage 2: Analyzer
```

```
Query Tree (semantic)
|
▼ Stage 3: Rewriter
Query Tree (rewritten)
|
▼ Stage 4: Planner
Plan Tree
|
▼ Stage 5: Executor
Result rows → client
```

Each stage transforms the input into a progressively more concrete representation.

Stage 1: Parser (Raw Parse Tree)

The **parser** takes the raw SQL text and converts it into a **parse tree** (also called the raw parse tree) using a hand-written grammar (Bison/Flex).

At this stage:

- SQL syntax is validated.
- No catalog lookups are done.
- Object names are stored as strings.
- No type resolution yet.

The result is a `List` of `RawStmt` nodes, each containing a statement-specific node type (e.g., `SelectStmt`, `InsertStmt`, `CreateTableStmt`).

Errors at this stage:

- Syntax errors: `SELECT * FORM users` → `ERROR: syntax error at or near "FORM"`.

```
-- To see the raw parse tree (PostgreSQL internal developer function)
SELECT pg_parse_query('SELECT id, name FROM users WHERE id = 1');
-- Returns an internal text representation (not user-facing in production)
```

Stage 2: Analyzer / Semantic Analysis (Query Tree)

The **analyzer** (also called semantic analysis) converts the raw parse tree into a **query tree** (`Query` struct) by:

1. **Name resolution** — Looking up table names, column names, function names in `pg_class`, `pg_attribute`, `pg_proc`.
2. **Type resolution** — Determining the data type of each expression.
3. **Subquery flattening** — Converting some subqueries into joins (simple cases).
4. **System column injection** — Adding system columns like `tableoid`, `ctid` if referenced.
5. **Permission checking** — Verifying the current user has the necessary privileges.

The output is a `Query` struct containing:

- `rtable` (range table) — list of all relations referenced
- `jointree` — the join structure
- `targetList` — the output expressions

- `qual` — the WHERE clause expression tree

Errors at this stage:

- `ERROR: relation "usres" does not exist` — misspelled table name.
- `ERROR: column "usr_id" does not exist`.
- `ERROR: permission denied for table secret`.

Stage 3: Rewriter (Rules and Views)

The **rewriter** applies PostgreSQL's rule system to the query tree. Rules are stored in `pg_rewrite`.

The primary use of the rewriter:

- **View expansion** — A `SELECT` against a view is rewritten to a `SELECT` against the view's underlying query. The view definition is stored as a rule in `pg_rewrite`.
- **DO INSTEAD rules** — Custom rules that replace the original statement with a different one.
- **DO ALSO rules** — Custom rules that add additional statements.

After the rewriter, the query tree reflects the fully expanded SQL — as if views had never existed.

```
-- See the rewrite rules for a view
SELECT * FROM pg_rewrite WHERE ev_class = 'my_view'::regclass;

-- Show view definition (the rule body)
SELECT pg_get_viewdef('my_view', true);
```

Stage 4: Planner / Optimizer

The **planner** (optimizer) takes the rewritten query tree and produces a **plan tree** — a tree of plan nodes that the executor will execute. The planner's goal is to find the lowest-cost execution plan.

Cost Estimation

Costs are expressed in abstract units where reading one 8 KB page from disk = `seq_page_cost` (default 1.0).

Key cost parameters:

Parameter	Default	Meaning
<code>seq_page_cost</code>	1.0	Cost to read one page sequentially (baseline)
<code>random_page_cost</code>	4.0	Cost to read one page randomly (disk seek)
<code>cpu_tuple_cost</code>	0.01	Cost to process one tuple
<code>cpu_index_tuple_cost</code>	0.005	Cost to process one index entry
<code>cpu_operator_cost</code>	0.0025	Cost to evaluate one operator/function
<code>effective_cache_size</code>	4 GB	Planner's estimate of OS page cache size (affects cost of random I/O)

The planner computes:

- **Startup cost** — cost before the first row can be returned (e.g., sort, hash build).
- **Total cost** — cost to produce all rows.
- **Row estimate** — estimated number of rows the plan node will produce.

These estimates come from `pg_statistic` (column statistics from ANALYZE).

Join Ordering

For queries joining N tables, the planner must choose the join order. The search space grows as $O(N!)$ — for N=10 this is 3.6 million possible orders.

PostgreSQL uses:

- **Dynamic programming** — exhaustive search for $N \leq \text{join_collapse_limit}$ (default 8).
- **Genetic query optimizer (GEQO)** — probabilistic search for $N > \text{geqo_threshold}$ (default 12).

Index Selection

The planner considers all applicable indexes and estimates:

1. **Index scan cost** = (index pages) × `random_page_cost` + (heap pages fetched) × `random_page_cost`
2. **Sequential scan cost** = (table pages) × `seq_page_cost`

It also considers:

- **Bitmap heap scan** — gather all matching TIDs from the index, sort them, fetch heap pages in disk order (much better than random index scan for large result sets).
- **Index-only scan** — if the visibility map allows, avoid heap entirely.
- **Parallel scan** — for large tables, use multiple workers.

Key Planner Parameters

Parameter	Default	Effect when disabled
<code>enable_seqscan</code>	on	off = planner avoids sequential scans (adds huge cost penalty)
<code>enable_indexscan</code>	on	off = avoids regular index scans
<code>enable_indexonlyscan</code>	on	off = avoids index-only scans
<code>enable_bitmapscan</code>	on	off = avoids bitmap heap scans
<code>enable_sort</code>	on	off = avoids explicit Sort nodes
<code>enable_hashjoin</code>	on	off = avoids hash join (uses merge join or nested loop)
<code>enable_mergejoin</code>	on	off = avoids merge join
<code>enable_nestloop</code>	on	off = avoids nested loop join
<code>enable_hashagg</code>	on	off = avoids hash aggregate
<code>enable_material</code>	on	off = avoids materialise nodes
<code>enable_partitionwise_join</code>	off	on = allow joining partitions directly
<code>enable_parallel_hash</code>	on	off = disables parallel hash join

Warning: Disabling planner parameters (e.g., `SET enable_seqscan = off`) in production is a **last resort** workaround for bad plans. The real fix is updated statistics or adjusted GUC costs.

Stage 5: Executor

The **executor** implements a **volcano (iterator) model**: each plan node has three operations:

- `ExecInitNode` — initialise the node.
- `ExecProcNode` — return the next tuple (called repeatedly by the parent node).
- `ExecEndNode` — clean up.

Execution is **pull-based**: the top node calls the child's `ExecProcNode`, which calls its child, and so on. Rows "flow up" the tree one at a time (or in batches with vectorised execution in PostgreSQL 17+ partial).

Execution context

The executor maintains:

- **EState** (executor state) — transaction snapshot, result relation info, parameter bindings.
 - **PlanState** — per-node runtime state (hash tables, sort buffers, scan position).
-

Plan Node Types

Scan nodes (leaf nodes, read data)

Node	Description
<code>SeqScan</code>	Sequential heap scan
<code>IndexScan</code>	Index scan + heap fetch per row
<code>IndexOnlyScan</code>	Index scan only (no heap if VM says visible)
<code>BitmapIndexScan</code>	Build bitmap of TIDs from index
<code>BitmapHeapScan</code>	Fetch heap pages from TID bitmap
<code>TidScan</code>	Fetch specific TIDs directly (e.g., <code>WHERE ctid = '(0,1)'</code>)
<code>FunctionScan</code>	Scan result of set-returning function
<code>ValuesScan</code>	Scan a VALUES list
<code>ForeignScan</code>	Foreign data wrapper scan

Join nodes

Node	Description
<code>NestedLoop</code>	For each outer row, scan the inner. Fast for small inputs or indexed inner.
<code>HashJoin</code>	Build hash table on smaller side, probe with larger side. Best for large unsorted joins.
<code>MergeJoin</code>	Both inputs must be sorted on join key. Efficient for pre-sorted or indexed inputs.

Aggregate / grouping nodes

Node	Description
HashAggregate	Hash-based GROUP BY. Fast, requires memory.
GroupAggregate	Sort-based GROUP BY. Requires sorted input.
Aggregate	Simple aggregate without GROUP BY.

Utility nodes

Node	Description
Sort	Sort tuples (in-memory or on-disk with <code>work_mem</code>)
Limit	Apply LIMIT/OFFSET
Append	Union of multiple subplans (UNION ALL, partition pruning)
Materialize	Buffer all rows in memory for repeated access
Result	Evaluate a constant expression
Unique	Remove duplicates from sorted input (for DISTINCT)
WindowAgg	Window functions
Gather	Collect results from parallel workers
GatherMerge	Collect pre-sorted results from parallel workers

EXPLAIN and EXPLAIN ANALYZE

```
-- Basic plan (estimated costs only)
EXPLAIN SELECT o.id, c.name FROM orders o JOIN customers c ON c.id = o.customer_id;

-- Actual timing (runs the query)
EXPLAIN (ANALYZE) SELECT ...;

-- Full detail with buffers
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ...;

-- JSON format for tooling
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...;
```

Reading EXPLAIN output

```
Hash Join  (cost=5.25..20.48 rows=100 width=48)
    actual rows=95 loops=1
    Hash Cond: (o.customer_id = c.id)
    -> Seq Scan on orders o  (cost=0.00..10.00 rows=1000 width=20)
```

```

                                actual rows=1000 loops=1
-> Hash  (cost=3.00..3.00 rows=180 width=32)
                                actual rows=180 loops=1
                                Buckets: 256  Batches: 1  Memory Usage: 24kB
-> Seq Scan on customers c  (cost=0.00..3.00 rows=180 width=32)

```

Key fields:

- `cost=startup..total` — estimated costs.
- `rows=N` — estimated row count.
- `width=N` — estimated bytes per output row.
- `actual rows=N loops=N` — with ANALYZE: actual rows returned, loops executed.
- `Buffers: shared hit=N read=N` — with BUFFERS: cache hits vs disk reads.

Misestimation patterns

Observation	Likely Cause	Fix
rows=1 but actual rows=100000	Stale statistics or correlation mismatch	ANALYZE; adjust <code>default_statistics_target</code>
Seq scan on large table	Missing index or low <code>seq_page_cost</code> VS <code>random_page_cost</code>	Create index; check cost params
Nested loop with large outer	Wrong row estimate → bad join order	ANALYZE; consider <code>set enable_nestloop=off</code> as temp fix
Sort spilling to disk	<code>work_mem</code> too low	Increase <code>work_mem</code> per session
Many Buffers: read=	Cold cache or poor index selectivity	Warm cache; check index column selectivity

ASCII Pipeline Diagram

SQL Text

```
"SELECT id, name FROM orders o JOIN customers c ON c.id = o.customer_id WHERE
o.status = 'shipped'"
|
```

▼ PARSER (lex + grammar, no catalog access)

RawStmt → SelectStmt

```
|— fromClause: [JoinExpr(o, c)]
|— targetList: [ColumnRef(id), ColumnRef(name)]
|— whereClause: A_Expr(status = 'shipped')
|
```

▼ ANALYZER (catalog lookups, type resolution)

Query

```
|— rtable: [{o: orders OID 16385}, {c: customers OID 16390}]
|— jointree: JoinExpr(INNER, o, c, c.id = o.customer_id)
|— targetList: [Var(o.id int8), Var(c.name text)]
|— qual: OpExpr(=, Var(o.status), Const('shipped'))
|
```

▼ REWRITER (view expansion, rules – no-op if no views/rules)

Query (potentially modified if views used)

|

▼ PLANNER / OPTIMIZER

Plan Tree:

HashJoin (cost=5.25..20.48 rows=100)

Hash Cond: c.id = o.customer_id

├─ SeqScan on orders o (cost=0.00..10.00)

│ Filter: status = 'shipped'

└─ Hash

└─ SeqScan on customers c (cost=0.00..3.00)

|

▼ EXECUTOR (volcano / iterator model)

Calls HashJoin.ExecProcNode()

→ calls SeqScan(orders).ExecProcNode() → reads heap pages

→ calls SeqScan(customers).ExecProcNode() → builds hash table

→ probes hash table for each orders row

→ returns matching tuples one by one

|

▼ RESULT

Rows sent to client

Live Observation Queries

```
-- 1. Force a specific plan node type for testing
SET enable_hashjoin = off;
SET enable_nestloop = off;
EXPLAIN SELECT o.id FROM orders o JOIN customers c ON c.id = o.customer_id;
RESET enable_hashjoin;
RESET enable_nestloop;

-- 2. Verbose EXPLAIN with all metadata
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT TEXT)
SELECT * FROM orders WHERE customer_id = 1 ORDER BY created_at DESC LIMIT 10;

-- 3. Look up the rewrite rules for a view
SELECT rulename, ev_type, pg_get_ruledef(oid, true) AS definition
FROM pg_rewrite
WHERE ev_class = 'order_summary'::regclass;

-- 4. Check planner statistics for a column
SELECT tablename, attname, n_distinct, correlation,
       array_length(most_common_vals, 1) AS mcv_count
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'status';

-- 5. Increase statistics target for a column
ALTER TABLE orders ALTER COLUMN status SET STATISTICS 500;
ANALYZE orders;

-- 6. Check current plan-enabling GUCs
```



```

SELECT name, setting
FROM pg_settings
WHERE name LIKE 'enable_%'
ORDER BY name;

-- 7. Show planner cost parameters
SELECT name, setting, unit
FROM pg_settings
WHERE name IN (
    'seq_page_cost', 'random_page_cost',
    'cpu_tuple_cost', 'cpu_index_tuple_cost',
    'cpu_operator_cost', 'effective_cache_size',
    'work_mem', 'hash_mem_multiplier',
    'join_collapse_limit', 'from_collapse_limit',
    'geqo', 'geqo_threshold', 'geqo_effort'
);

-- 8. Find queries with the most planning time (pg_stat_statements)
SELECT query, calls, mean_plan_time, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY mean_plan_time DESC
LIMIT 10;

-- 9. Observe the physical plan for an aggregate
EXPLAIN (ANALYZE, BUFFERS)
SELECT status, count(*), sum(total) FROM orders GROUP BY status;
-- Look for: HashAggregate vs GroupAggregate, Sort nodes

-- 10. Test index-only scan condition
EXPLAIN (ANALYZE)
SELECT id FROM orders WHERE id BETWEEN 1 AND 1000;
-- Look for: Index Only Scan, Heap Fetches count

```

Common Mistakes

- Disabling planner nodes without understanding root cause.** `SET enable_seqscan = off` is a workaround, not a fix. Always investigate why the planner chose the bad plan — usually stale statistics or incorrect cost parameters.
- Not running ANALYZE after large data changes.** The planner uses `pg_statistic` for estimates. After a bulk load, `ANALYZE` must be run or the planner will use wildly wrong row estimates.
- Misinterpreting cost= in EXPLAIN.** The cost values are unitless estimates, not milliseconds. Compare plans against each other; do not try to infer wall-clock time from costs.
- Ignoring loops=N in EXPLAIN ANALYZE.** Actual rows shown is "per loop". If loops=1000 and actual rows=10, the node produces 10,000 rows total. The planner estimates "per loop" too.
- Over-trusting row estimates.** For complex queries with many joins and filters, row estimates can be off by orders of magnitude. Always cross-check with actual rows in EXPLAIN ANALYZE.

6. Forgetting that **EXPLAIN ANALYZE** actually executes the query. For `INSERT / UPDATE / DELETE` , wrap in a transaction and rollback if you just want to see the plan without side effects.

Best Practices

- Run `EXPLAIN (ANALYZE, BUFFERS)` rather than plain `EXPLAIN` when diagnosing performance — actual numbers are far more useful than estimates.
- Run `ANALYZE` after bulk loads to give the planner fresh statistics.
- Increase `default_statistics_target` (e.g., to 200) for columns with high cardinality or skewed distributions.
- Set `random_page_cost = 1.1` on systems with fast SSDs (the 4.0 default is calibrated for spinning disk).
- Tune `work_mem` per session for complex queries rather than globally — a high global `work_mem` multiplied by many sessions can exceed available RAM.
- Use `pg_stat_statements` to identify high-planning-time queries — they may benefit from `plan_cache_mode = force_generic_plan` .

Performance Considerations

Issue	Symptom	Fix
Wrong join order	Nested loop with large outer rows	ANALYZE; increase statistics target for join columns
Seq scan on large table	Slow full-table scan	Create appropriate index
Sort spilling to disk	"Sort Method: external merge Disk: Nkb" in EXPLAIN ANALYZE	Increase <code>work_mem</code> for this session/query
Hash join batches > 1	"Batches: N" in hash plan	Increase <code>work_mem</code>
Parallel scan not used	Large table, slow sequential scan	Check <code>max_parallel_workers_per_gather</code> , <code>min_parallel_table_scan_size</code>
Planning time > execution time	Many OR conditions, complex joins	Rewrite query; use CTEs; increase plan cache use
Stale statistics	Large row count discrepancy	ANALYZE; increase <code>autovacuum_analyze_scale_factor</code>

Interview Questions

Q1. What are the five stages of PostgreSQL query execution?

Parser (SQL text → raw parse tree), Analyzer (raw parse tree → query tree with catalog lookups and type resolution), Rewriter (view expansion, rule application), Planner/Optimizer (query tree → plan tree with cost estimation), Executor (plan tree → result rows via volcano/iterator model).

Q2. What does the analyzer do that the parser does not?

The analyzer performs catalog lookups (resolves table names, column names, function names to their OIDs and types), type resolution, permission checking, and subquery transformations. The parser only validates SQL syntax without accessing the catalog.

Q3. What is the volcano/iterator model used by the PostgreSQL executor?

Each plan node implements a `GetNext()` (`ExecProcNode`) interface. The top node calls its child's `GetNext()` to retrieve the next tuple. This propagates down the tree. Execution is pull-based and lazy — rows are produced on demand. This allows `LIMIT` to work without evaluating the full result set.

Q4. How does the planner estimate the number of rows a SeqScan will return?

It uses column statistics from `pg_statistic` (populated by `ANALYZE`): the `n_distinct` count, `most_common_vals` with frequencies, and `histogram_bounds`. For a filter like `status = 'shipped'`, it checks if 'shipped' is in `most_common_vals` and uses its frequency, or falls back to `1/n_distinct`. The result is multiplied by `pg_class.rel tuples`.

Q5. When would a Hash Join be preferred over a Nested Loop?

Hash Join is preferred when both input relations are large and neither is indexed on the join column. It builds a hash table on the smaller relation ($O(N)$) and probes with the larger ($O(M)$). Nested Loop is better when the outer relation is small or the inner has an index.

Q6. What does `SET enable_seqscan = off` do and when should you use it?

It adds a high penalty to the cost of sequential scans, effectively forcing the planner to prefer index scans. Use only for diagnostic purposes or as a temporary workaround when the planner is choosing a seq scan due to bad statistics. The real fix is updating statistics with `ANALYZE` or correcting cost parameters.

Q7. What is the difference between `cost startup` and `total` in `EXPLAIN`?

Startup cost is the estimated cost before the first row can be returned (e.g., building a hash table, sorting all input). Total cost is the estimated cost to produce all output rows. A node with high startup and low total is efficient once started (e.g., Sort); a node with zero startup is a streaming node (e.g., SeqScan).

Q8. What causes "Heap Fetches" in an Index Only Scan?

The visibility map (VM) bit for the heap page is not set (all-visible bit is 0). The executor must fetch the heap page to verify tuple visibility. Run `VACUUM` on the table to set the VM bits and eliminate heap fetches.

Q9. What does the rewriter do to a `SELECT * FROM my_view`?

It looks up the view's rule in `pg_rewrite`, substitutes the view's definition (the subquery) in place of the view reference in the query tree. The result is a query tree that references only base tables, as if the user had written the full view definition inline.

Q10. How does `work_mem` affect the executor?

`work_mem` is the per-operation memory limit for Sort nodes and hash table operations. Each Sort or Hash Join can use up to `work_mem` bytes. If the data exceeds `work_mem`, Sort uses a disk-based merge sort and Hash Join uses multiple batches (slower). Setting it too high wastes RAM when many queries run concurrently.

Exercises with Solutions

Exercise 1 — Read `EXPLAIN` output

```
-- Solution: create test data and practice reading plan output
CREATE TABLE products (id serial PRIMARY KEY, category text, price numeric);
CREATE TABLE order_items (order_id int, product_id int, qty int);

INSERT INTO products SELECT i, 'cat_' || (i%10), random()*100
FROM generate_series(1,10000) i;
INSERT INTO order_items SELECT (random()*1000+1)::int, (random()*10000+1)::int,
(random()*10+1)::int
FROM generate_series(1,100000);

ANALYZE products; ANALYZE order_items;

-- Now examine different plan options
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.category, sum(oi.qty * p.price) AS revenue
FROM order_items oi
JOIN products p ON p.id = oi.product_id
GROUP BY p.category;

-- Questions to answer from the output:
-- 1. What join method was used? Why?
-- 2. What aggregation method was used?
-- 3. How accurate were the row estimates?
-- 4. How many buffer pages were read?
```

Exercise 2 — Force different join algorithms

```
-- Solution: compare Hash Join vs Merge Join vs Nested Loop

-- Hash Join
SET enable_mergejoin = off; SET enable_nestloop = off;
EXPLAIN (ANALYZE) SELECT ... FROM order_items oi JOIN products p ON p.id =
oi.product_id;

-- Merge Join (require sorted input)
RESET enable_mergejoin; SET enable_hashjoin = off; SET enable_nestloop = off;
EXPLAIN (ANALYZE) SELECT ...;

-- Nested Loop
RESET ALL;
SET enable_hashjoin = off; SET enable_mergejoin = off;
EXPLAIN (ANALYZE) SELECT ...;

-- Compare actual total times across all three
RESET ALL;
```

Exercise 3 — Statistics and plan quality

```

-- Solution: show how stale statistics cause bad plans
CREATE TABLE skewed (id serial, type_code int);
-- Insert highly skewed data: 99% have type_code=1, 1% have type_code=2
INSERT INTO skewed SELECT i,
    CASE WHEN i % 100 = 0 THEN 2 ELSE 1 END
FROM generate_series(1,100000) i;

ANALYZE skewed;

-- Show statistics
SELECT attname, most_common_vals, most_common_freqs, n_distinct
FROM pg_stats WHERE tablename = 'skewed' AND attname = 'type_code';

-- Plan for the rare value (good estimate expected)
EXPLAIN SELECT * FROM skewed WHERE type_code = 2;
-- Expected: rows ~1000 actual: ~1000 ✓

-- Corrupt statistics by deleting the analyze result and querying
-- (In practice, stale stats develop naturally from INSERT without ANALYZE)

```

Cross-References

- **10_system_catalogs.md** — `pg_statistic`, `pg_stats`, `pg_rewrite`, `pg_proc` used by pipeline
- **07_Indexes** — Index types and when the planner chooses them
- **08_Query_Optimization** — EXPLAIN analysis, join tuning, statistics
- **02_heap_storage.md** — SeqScan reads heap pages described here
- **08_visibility_map.md** — Index-only scan uses VM to skip heap fetches